

Day 6 — Strings Basics in Python

Detailed Seminar Notes | Definition • Indexing • Slicing • String Methods • Rules • Examples • Viva

According to your Python roadmap, **Day 6 = Strings Basics** with focus on **indexing, slicing, and methods**. Strings are one of the most important data types in Python because text data appears everywhere: names, messages, passwords, file content, sentences, and user input. In this day, the goal is to understand what strings are, how characters are stored, how to access them using indexing, how to extract parts using slicing, and how to use common string methods.

1) What is a String?

A **string** is a sequence of characters enclosed in **single quotes**, **double quotes**, or triple quotes. A character can be a letter, number, symbol, or even a space.

Examples of strings:

```
name = "Adi"
college = 'BVC'
message = "Python is easy"
```

In all the above examples, the values are strings because they are written inside quotes.

2) Why are strings important?

- Strings are used to store **names, addresses, passwords, messages, sentences, and file text**.
- Most user input in Python is received as a string by default.
- Strings are used in **websites, forms, chat apps, data processing, and automation**.
- Learning strings helps in solving many beginner problems like reverse a word, count letters, check palindrome, replace text, etc.

3) Creating strings in Python

Strings can be created using:

Type	Example
Single quotes	name = 'Adi'
Double quotes	city = "Yanam"
Triple quotes	text = """Python is beginner friendly"""

Notes

- Single quotes and double quotes are commonly used for one-line strings.
- Triple quotes are useful for multi-line text.
- The opening and closing quotes must match.

4) String is a sequence of characters

A string stores characters in order. Each character has a **position** called an **index**.

```
word = "PYTHON"
```

Here the characters are stored as:

Character	P	Y	T	H	O	N
Positive index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

5) String Indexing

Indexing means accessing a **single character** from a string using its position.

Syntax:

```
string_name[index]
```

Examples of indexing

```
word = "PYTHON"

print(word[0])    # P
print(word[1])    # Y
print(word[5])    # N
print(word[-1])   # N
print(word[-2])   # O
```

Explanation

- Positive indexing starts from **0** on the left side.
- Negative indexing starts from **-1** on the right side.
- If the index is out of range, Python gives an error called **IndexError**.

Example of IndexError

```
word = "PYTHON"
print(word[10])
```

This gives an error because the string has only 6 characters, so index 10 does not exist.

6) String Slicing

Slicing means taking a **part of a string** instead of a single character.

General syntax:

```
string_name[start : stop : step]
```

Where:

- **start** → index where slicing begins
- **stop** → index where slicing stops (this value is **not included**)
- **step** → jump value

Example string

```
text = "PYTHON"
```

Slicing examples

Expression	Result	Reason
------------	--------	--------

text[0:3]	PYT	characters from index 0 to 2
text[1:4]	YTH	characters from index 1 to 3
text[:4]	PYTH	from beginning to index 3
text[2:]	THON	from index 2 to end
text[-4:-1]	THO	negative slicing
text[::2]	PTO	every 2nd character

Important slicing rules

- The **stop index is not included**.
- If start is omitted, Python starts from the beginning.
- If stop is omitted, Python goes till the end.
- If step is omitted, the default step is 1.
- Negative indices can also be used in slicing.

7) Reverse a string using slicing

A very popular string trick in Python is reversing a string using `[::-1]`.

```
text = "python"
print(text[::-1])
```

Output: nohtyp

Here, the step is **-1**, so Python moves from the end to the beginning.

8) Strings are immutable

A very important property of strings is that they are **immutable**. Immutable means once a string is created, its characters **cannot be changed directly**.

```
name = "Adi"
name[0] = "a"
```

The above code gives an error because strings do not allow direct character modification.

Correct way

```
name = "Adi"
name = "a" + name[1:]
print(name)
```

This creates a **new string** instead of changing the old one.

9) Common String Methods

String methods are built-in functions that help us perform operations on strings.

Method	Purpose	Example
upper()	Converts to uppercase	"python".upper() → "PYTHON"
lower()	Converts to lowercase	"PYTHON".lower() → "python"
title()	Converts first letter of each word to uppercase	"python basics".title()
capitalize()	Capitalizes first letter of the string	"python".capitalize()
strip()	Removes spaces from both ends	" hi ".strip()

replace(old, new)	Replaces text	"python".replace("py","my")
find(x)	Returns index of first occurrence	"python".find("t")
count(x)	Counts occurrences	"banana".count("a")
startswith(x)	Checks starting text	"python".startswith("py")
endswith(x)	Checks ending text	"python".endswith("on")

10) Examples of string methods

```
text = "  python basics  "

print(text.upper())      # "  PYTHON BASICS  "
print(text.lower())      # "  python basics  "
print(text.title())      # "  Python Basics  "
print(text.strip())      # "python basics"
print(text.replace("python", "java")) # "  java basics  "
```

More examples

```
word = "banana"

print(word.count("a"))   # 3
print(word.find("n"))    # 2
print(word.startswith("ba")) # True
print(word.endswith("na")) # True
```

11) len() function with strings

The **len()** function returns the total number of characters in a string, including spaces.

```
text = "Python"
print(len(text))
```

Output: 6

```
msg = "Hi Adi"
print(len(msg))
```

Output is 6 because the space is also counted.

12) Membership operators with strings

We can check whether a character or word exists inside a string using **in** and **not in**.

```
text = "python"

print("py" in text)      # True
print("z" in text)      # False
print("thon" in text)    # True
print("java" not in text) # True
```

13) Traversing a string

Traversing means visiting each character one by one.

Using for loop

```
word = "PYTHON"

for ch in word:
    print(ch)
```

Using index

```
word = "PYTHON"

for i in range(len(word)):
    print(word[i])
```

14) Comparing strings

Strings can be compared using **==**, **!=**, and sometimes alphabetically using **<** or **>**.

```
a = "python"
b = "python"
c = "java"

print(a == b)  # True
print(a == c)  # False
print(a != c)  # True
```

15) Important beginner string programs

Program 1: Print each character of a string

```
text = "Python"
for ch in text:
    print(ch)
```

Program 2: Count vowels in a string

```
text = input("Enter a string: ")
count = 0

for ch in text.lower():
    if ch in "aeiou":
        count += 1

print("Vowels =", count)
```

Program 3: Check palindrome

```
text = input("Enter a word: ")

if text == text[::-1]:
```

```

        print("Palindrome")
    else:
        print("Not palindrome")

```

Program 4: Reverse a string

```

text = input("Enter a string: ")
print(text[::-1])

```

16) Common beginner mistakes with strings

- Confusing **indexing** with **slicing**.
- Forgetting that slicing stop value is not included.
- Trying to change a character directly in a string, even though strings are immutable.
- Using an index that is out of range.
- Thinking **len()** ignores spaces — it does not.
- Forgetting that many string methods return a **new string**.

17) How to explain Day 6 in a seminar

- Start with the definition of a string and give 2–3 simple examples.
- Explain that a string is a sequence of characters and show the index table for a word like **PYTHON**.
- Teach **indexing** first, then **slicing**.
- Show the difference between **text[0]** and **text[0:3]**.
- Explain **immutable** in a simple way.
- Teach common methods like **upper()**, **lower()**, **strip()**, **replace()**, **count()**, and **find()**.
- End with small programs like reverse string, palindrome, and vowel count.

18) Viva / seminar questions

- What is a string in Python?
- How are strings written in Python?
- What is indexing?
- What is the difference between positive and negative indexing?
- What is slicing? Why is the stop value not included?
- What does **text[::-1]** do?
- What does immutable mean in strings?
- What is the use of **len()**?
- What is the difference between **find()** and **count()**?
- Name any 5 string methods.

19) Short seminar summary

Strings are sequences of characters used to store text in Python. Important beginner concepts are **indexing**, **slicing**, **len()**, **membership operators**, and common **string methods**. Strings are **immutable**, so characters cannot be changed directly. Mastering strings is essential because they are used in user input, messages, files, and many real-world programs.

20) Quick revision box

Topic	Key point
String	Sequence of characters written inside quotes
Indexing	Used to access one character using its position
Slicing	Used to extract a part of a string
Negative index	Counts from right side, starting at -1
Immutable	String characters cannot be changed directly
len()	Returns number of characters in the string
Common methods	upper(), lower(), strip(), replace(), count(), find()